AM5630 - FOUNDATION OF COMPUTATIONAL FLUID DYNAMICS

Karthick.j
OE23S018
M.S SCHOLAR
DEPARTMENT OF OCEAN ENGINEERING
IIT MADRAS,
CHENNAI

Problem Definition:

Consider a rectangular plate with dimensions 1 x 2 units and unit thickness. Determine the steady state temperature distribution within the plate under the given boundary conditions.

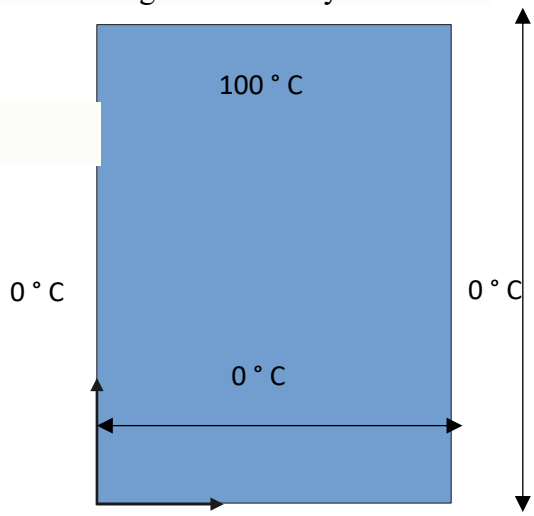
Governing Equation:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Boundary Conditions:

Top wall = 100 º C

All other walls = 0º



Numerical formulation:

Point Gauss seidel method:

$$T(ij)^{K+1} = \frac{1}{2(1+\beta^2}\ [\ T^k_{i+1j} + T^k_{i-jj} + \beta^2 T^k_{ij-1} + \beta^2 T^{k+1}_{ij-1}$$

Line gauss Seidel method

$$T^{k+1}_{i-1j} - 2(1+\beta^2)T^{k+1}_{ij} + T^{k+1}_{i+1j} = -\beta^2(T^k_{ij+1} + T^{k+1}_{ij-1})$$

Point successive over relaxation method:

$$T(ij)^{K+1} = (1-\omega)T(ij)^K + \frac{1}{2(1+\beta^2}\ [\ T^k_{i+1j} + T^k_{i-jj} + \beta^2 T^k_{ij-1} + \beta^2 T^{k+1}_{ij-1}$$

Line successive over relaxation method:

$$\omega T^{k+1}_{i-1j} - 2(1+\beta^2)T^{k+1}_{ij} + \omega T^{k+1}_{i+1j} = (1-\omega)\ 2(1+\beta^2)T(ij)^K + \omega\beta^2(T^k_{ij+1} + T^{k+1}_{ij-1})$$

ADI method

$$T^{k+1/2}_{i-1j} - 2(1+\beta^2)T^{k+1/2}_{ij} + T^{k+1/2}_{i+1j} = -\beta^2(T^{k+1/2}_{ij+1} + T^k_{ij-1})$$

$$-(T^{k+1}_{i-1j} - + T^{k+1}_{i+1j}) = \beta^2\left(T^k_{ij+1} + T^{k+1}_{ij-1}\right) - 2(1+\beta^2)T^{k+1}_{ij}$$

ADI method With Relaxation Factor:

$$\omega T^{k+1/2}_{i-1j} - 2(1+\beta^2)T^{k+\frac{1}{2}}_{ij} + \omega T^{k+\frac{1}{2}}_{i+1j} = -(1-\omega)\ 2(1+\beta^2)T(ij)^K - \omega\beta^2(T^{k+1/2}_{ij-1} + T^k_{ij+1})$$

$$\omega\beta^2 T^{k+1}_{i-1j} - 2(1+\beta^2)T(ij)^{K+\frac{1}{2}} + \beta^2\omega T^{k+1}_{i+1j} = -(1-\omega)\ 2(1+\beta^2)T(ij)^{K+\frac{1}{2}} + \omega(T^k_{ij+1} + T^{k+1/2}_{ij-1}$$

1.ALGORITHM FOR POINT GAUSS-SIDEL METHOD:

1. Initialize the grid size with all zero values.
2. Calculate the cell size in X and Y axes and the beta value.
3. Set the boundary conditions using the Boundary_condition() function.
4. Start the iterative process with the first iteration:
   a. Loop over all interior grid points (excluding the boundary points) and calculate the new temperature value using the finite difference method.
   b. Set the boundary conditions again.
5. Set the tolerance level and the iteration counter to zero.
6. Start the main iteration loop:
   a. Loop over all interior grid points (excluding the boundary points) and calculate the new temperature value using the finite difference method.
   b. Calculate the difference between the old and new temperature values.
   c. If the difference is greater than the tolerance level, increment the tolerance not reached counter and break out of the loop.
   d. If the tolerance not reached counter is zero, break out of the loop.
   e. If the tolerance not reached counter is greater than zero, update the temperature grid with the new values and write the results to the output file.
7. Print the number of iterations taken to reach the tolerance level and the elapsed CPU time.

2.ALGORITHM FOR LINE GAUSS- SIDEL METHOD:

Initialization:

- Define constants and grid dimensions (NX, NY).
- Set boundary temperatures (T_wall and T_top_wall).
- Initialize temperature grid T and other parameters.

Boundary Conditions Setup (Boundary_condition):

- Apply boundary conditions to the grid (T_wall on the left, right, and bottom; T_top_wall on the top).

First Iteration:

- Perform the first iteration of the Gauss-Seidel method to initialize values in the grid based on the boundary conditions.

Iterative Solution (while(true) loop):

- Define a tolerance level (tolerance) to check convergence.
- Initialize a loop to perform iterative updates until the solution converges (changes are within the tolerance level).
- For each iteration:
- Calculate new temperatures (T_new) using the Gauss-Seidel method.
- Check the change in temperatures against the tolerance level to determine convergence.
- If converged, exit the loop; otherwise, update T with T_new values.

Output Results:

- Write the final temperature distribution (T) to an output file (final_temp.txt).
- Measure and display the total number of iterations required for convergence and the CPU time used.

3.ALGORITHM FOR POINT SUCCESSIVE OVER RELAXATION METHOD:

1. Initialize the grid size and parameters:
2. Set the grid dimensions NX and NY.
3. Define the domain size LX and LY.
4. Set the boundary temperatures T_top_wall and T_wall.
5. Initialize the temperature grid T, cell sizes delta_x and delta_y, beta, and relaxation parameter omega.
6. Define the Boundary_condition() function:
7. Set the boundary conditions for the temperature grid.
8. Main function:
9. Start the clock to measure CPU time.
10. Calculate delta_x, delta_y, and beta.
11. Initialize the temperature grid T with zero values.
12. First iteration:
13. Perform the first iteration using PSOR method:
14. Loop over all interior grid points (excluding boundaries).
15. Update the temperature at each point using the PSOR formula.
16. Set boundary conditions using Boundary_condition().
17. Iterative process:
18. Set tolerance level and initialize iteration counter and a new temperature grid T_new.
19. Open an output file for writing results.
20. Start the iterative loop:
21. Increment the iteration counter.
22. Perform iterative calculations:
23. Update the temperature grid using the PSOR formula.
24. Check if the desired tolerance level has been reached.
25. Update the temperature grid with new values.
26. Output results to a file and continue iterating until the tolerance level is reached.
27. Print th*e number of iterations taken to reach the tolerance level and the elapsed CPU time.*

4.ALGORITHM FOR LINE SUCCESSIVE OVER RELAXATION METHOD:

- Initialize the grid size and parameters:
- Set the grid dimensions NX and NY.
- Define the domain size LX and LY.
- Set the boundary temperatures T_top_wall and T_wall.
- Initialize the temperature grid T, cell sizes delta_x and delta_y, beta, and relaxation parameter omega.
- Define the Boundary_condition() function:
- Set the boundary conditions for the temperature grid.
- Main function:
- Start the clock to measure CPU time.
- Calculate delta_x, delta_y, and beta.
- Initialize the temperature grid T with zero values.
- Perform the first iteration using LSOR Gauss-Seidel method:
- Loop over the interior grid points along each line in the x and y directions.
- Update the temperature at each point using the LSOR Gauss-Seidel formula.
- Set boundary conditions using Boundary_condition().
- Iterative process:
- Set tolerance level and initialize iteration counter and a new temperature grid T_new.
- Open an output file for writing results.
- Start the iterative loop:
- Increment the iteration counter.
- Perform iterative calculations:
- Update the temperature grid using the LSOR Gauss-Seidel formula.
- Check if the desired tolerance level has been reached.
- Update the temperature grid with new values.
- Output results to a file and continue iterating until the tolerance level is reached.
- Print the number of iterations taken to reach the tolerance level and the elapsed CPU time.

5.ALGORITHM FOR ADI METHOD WITHOUT RELAXATION FACTOR:

- **Initialize Grid and Parameters:**

  Define constants such as grid dimensions (NX and NY), domain sizes (LX and LY), and boundary temperatures (T_wall and T_top_wall).

  Initialize the temperature grid T, cell sizes delta_x and delta_y, and compute the beta parameter beta (beta = delta_x / delta_y).

- **Define Boundary Conditions (Boundary_condition Function):**

  Set the boundary conditions for the temperature grid:

  Bottom (j=0), top (j=NY-1), left (i=0), and right (i=NX-1) walls with specified temperatures.

- **Main Function Execution:**

  Start measuring CPU time.

  Calculate cell sizes (delta_x, delta_y) and compute the beta parameter (beta).

  Initialize the temperature grid T with zero values.

  Apply boundary conditions using Boundary_condition.

- **Iterative Process (while(true) Loop):**

  Initialize the iteration counter (iteration) with 0.5 for the first iteration (ADI method starts with half-step iterations).

- **ADI Iterative Steps (Alternating Directions):**

  Step 1 (X-direction Implicit):

  Update the temperature grid T_new in the X-direction (i loop inside j loop) using the ADI formula.

  Step 2 (Y-direction Implicit):

  Update the temperature grid T_new in the Y-direction (j loop inside i loop) using the ADI formula.

- **Convergence Check:**

  Calculate the change in temperature (delta_T) between T and T_new.

  Check if all grid points meet the desired tolerance level (tolerance).

- **Update and Output Results:**

  If convergence criteria are met (all points within tolerance), exit the loop.

  Update the temperature grid T with T_new values.

Write the current state of T to an output file (final_temp.txt).

Finalize and Output Performance Metrics:

Print the number of iterations taken to reach convergence (iteration).

Calculate and print the elapsed CPU time (CPU_time_used).

6.ALGORITHM FOR ADI METHOD WITH RLAXATION PARAMETER:

- Initialize Grid and Parameters:
- Define constants such as grid dimensions (NX and NY), domain sizes (LX and LY), boundary temperatures (T_wall and T_top_wall), and relaxation factor (omega).
- Initialize the temperature grid T, cell sizes delta_x and delta_y, and compute the beta parameter beta (beta = delta_x / delta_y).
- Define Boundary Conditions (Boundary_condition Function):
- Set the boundary conditions for the temperature grid:
- Bottom (j=0): T[i] = T_wall
- Top (j=NY-1): T[i][NY-1] = T_top_wall
- Left (i=0): T[j] = T_wall
- Right (i=NX-1): T[NX-1][j] = T_wall
- Main Function Execution:
- Start measuring CPU time.
- Calculate grid parameters (delta_x, delta_y, beta, and omega).
- Initialize the temperature grid T with zero values.
- Apply boundary conditions using Boundary_condition.
- First Iteration (for Loops for X and Y Directions):
- Update temperatures at the boundary edges (j=1 and j=NY-2 for X-direction, i=1 and i=NX-2 for Y-direction) using the ADI method and relaxation factor (omega).
- Iterative Process (while(true) Loop):
- Increment the iteration counter (iteration) by 0.5.
- Perform ADI calculations for X-direction and Y-direction:
- X-direction:
- Update T_new values using the ADI formula and relaxation factor (omega).
- Y-direction:

- Update T_new values using the ADI formula and relaxation factor (omega).
- Check convergence by calculating the change in temperature (delta_T) between T and T_new.
- If all grid points meet the desired tolerance level (tolerance), exit the loop.
- Update and Output Results:
- Update the temperature grid T with T_new values.
- Write the current state of T to an output file (final_temp.txt).
- Finalize and Output Performance Metrics:
- Print the number of iterations taken to reach convergence (iteration).
- Calculate and print the elapsed CPU time (CPU_time_used).

Results:

S.No	Scheme Used	Grid Size	No. of. iterations	CPU times (s)	Total computing time
1	Point Gauss-Siedel	21x41	347	2.5	6 hours
2	Line Gauss-Siedel	21x41	97	1.2	8 hours
3	Point successive over relaxation	21x41	Running	2 days	4 days
4	Line Successive over relaxation	21x 41	Running	1.5 days	3 days
5	ADI with Relaxation factor	21 x41	71	1.47	1 day
6	ADI without Relaxation factor	21 x 41	Running	1 day	1 day

COMPUTER CODE FOR GAUSS- SIEDEL METHOD:

```
#include<iostream>
#include<math.h>
#include<fstream>
#include<time.h>
```

```
#define NX 21
#define NY 41

double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta;

void Boundary_condition()
{
   for (int i=0; i<NX; i++)
   {
      T[i][0]= T_wall;
   }
   for (int i=0; i < NX; i++)
   {
      for (int j=0; j < (NY-1); j++)
      {
         T[i][j]=T_wall;
      }
   }
   for (int i=0; i < (NX-1); i++)
   {
      for (int j=0; j < NY; j++)
      {
         T[i][j]=T_wall;
      }
   }
   for (int i=0; i < NX; i++)
   {
      T[i][NY-1]= T_top_wall;
   }
```

```
}

int main()
{
   clock_t start, end;
   double CPU_time_used;
   start = clock();
   delta_x = (LX/ NX);
   delta_y = (LY/ NY);
   beta = (delta_x / delta_y);

   std::cout << "Cell size in X axis: " << delta_x << std::endl;
   std::cout << "Cell size in Y axis: " << delta_y << std::endl;
   std::cout << "Beta: " << beta << std::endl;

   // Initialize the grid size
   for(int i=0; i< NX; i++)
   {
      for(int j=0; j<NY; j++)
      {
         T[i][j]=0;
      }
   }

   // first iteration begin
   for(int i=1; i< NX-1; i++)
   {
      for(int j=1; j<NY-1; j++)
      {
         T[i][j]=((1/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
         //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
      }
   }
```

```
    Boundary_condition();

    //iteration start
    double tolerance = 0.01;
    int iteration = 0;
    double T_new[NX][NY];

    std::ofstream ofile("final_temp.txt");

    while(true)
    {
        iteration++;

        // Perform iterative calculations here
        for(int i=1; i< NX-1; i++)
        {
            for(int j=1; j<NY-1; j++)
            {
                T_new[i][j]=((1/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
            }
        }

        // Check if the desired tolerance level has been reached
        int tolerance_not_reached_count = 0;
        for(int i=1; i< NX-1; i++)
        {
            for(int j=1; j<NY-1; j++)
            {
                double delta_T = fabs(T_new[i][j] - T[i][j]);
                if(delta_T > tolerance)
                {
                    tolerance_not_reached_count++;
```

```
            break;
          }
        }
        if(tolerance_not_reached_count > 0)
        {
          break;
        }
      }

      // If the tolerance level has been reached, break out of the loop
      if(tolerance_not_reached_count == 0)
      {
        break;
      }

      // Update the temperature grid with the new values
      for(int i=1; i< NX-1; i++)
      {
        for(int j=1; j<NY-1; j++)
        {
          T[i][j] = T_new[i][j];
          //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
          //ofile  << T[i]<< std::endl;
          //ofile  << T[j]<< std::endl;
          //ofile  << T[i][j]<< std::endl;
          ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
                  //std::string combined = "T[" + std::to_string(i) + "][" + std::to_string(j) + "] = " +
std::to_string(T[i][j]) + "\n";
        //ofile << combined;
        }
      }
    }

    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
```

```
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds
    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;
    return 0;
}
```

COMPUTER CODE FOR LINE GAUSS- SIDEL METHOD:

```
//Line Gauss-sidel
#include<iostream>
#include<math.h>
#include<fstream>
#include <ctime>
#define NX 21
#define NY 41

double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta;

void Boundary_condition()
{
    for (int i=0; i<NX; i++)
    {
        T[i][0]= T_wall;
    }
    for (int i=0; i < NX; i++)
    {
        for (int j=0; j < (NY-1); j++)
```

```
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < (NX-1); i++)
    {
        for (int j=0; j < NY; j++)
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < NX; i++)
    {
        T[i][NY-1]= T_top_wall;
    }
}

int main()
{
    double clock_t,start, end;
    double CPU_time_used;
    start = clock();
    delta_x = (LX/ NX);
    delta_y = (LY/ NY);
    beta = (delta_x / delta_y);



    std::cout << "Cell size in X axis: " << delta_x << std::endl;
    std::cout << "Cell size in Y axis: " << delta_y << std::endl;
    std::cout << "Beta: " << beta << std::endl;

    // Initialize the grid size
```

```
   for(int i=0; i< NX; i++)
   {
      for(int j=0; j<NY; j++)
      {
         T[i][j]=0;
      }
   }

   // first iteration begin
   for(int j=1; j< NY-1; j++)
   {

T[0][j]=((1/(2*(1+(beta*beta))))*((T[1][j])+(beta*beta)*(T[0][j+1])+(beta*beta)*(T[0][j-1]))
);
   }


   for(int i=1; i< NX-1; i++)
   {
      T[i][1]=((1/(2*(1+(beta*beta))))*((T[i+1][1])+(T[i-1][1])+(beta*beta)*(T[i][2])) );
      for(int j=2; j< NY-1; j++)
      {
         T[i][j]=((1/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
      }
      T[i][NY-2]=((1/(2*(1+(beta*beta))))*((T[i+1][NY-2])+(T[i-1][NY-
2])+(beta*beta)*(T[i][NY-3])) );
   }


   for(int j=1; j< NY-1; j++)
   {
      T[NX-1][j]=((1/(2*(1+(beta*beta))))*((T[NX-2][j])+(beta*beta)*(T[NX-
1][j+1])+(beta*beta)*(T[NX-1][j-1])) );
   }
```

```
    Boundary_condition();

    //iteration start
    double tolerance = 0.01;
    int iteration = 0;
    double T_new[NX][NY];

    std::ofstream ofile("final_temp.txt");

    while(true)
    {
        iteration++;

        // Perform iterative calculations here
        for(int j=1; j< NY-1; j++)
        {
T_new[0][j]=((1/(2*(1+(beta*beta))))*((T[1][j])+(beta*beta)*(T[0][j+1])+(beta*beta)*(T[0][j-1])) );
        }

        for(int i=1; i< NX-1; i++)
        {
            T_new[i][1]=((1/(2*(1+(beta*beta))))*((T[i+1][1])+(T[i-1][1])+(beta*beta)*(T[i][2])) );
            for(int j=2; j< NY-1; j++)
            {
                T_new[i][j]=((1/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
            }
            T_new[i][NY-2]=((1/(2*(1+(beta*beta))))*((T[i+1][NY-2])+(T[i-1][NY-2])+(beta*beta)*(T[i][NY-3])) );
        }
```

```
    for(int j=1; j< NY-1; j++)
    {
       T_new[NX-1][j]=((1/(2*(1+(beta*beta))))*((T[NX-2][j])+(beta*beta)*(T[NX-1][j+1])+(beta*beta)*(T[NX-1][j-1])) );
    }

    // Check if the desired tolerance level has been reached
    int tolerance_not_reached_count = 0;
    for(int i=0; i< NX; i++)
    {
       for(int j=0; j<NY; j++)
       {
          double delta_T = fabs(T_new[i][j] - T[i][j]);
          if(delta_T > tolerance)
          {
             tolerance_not_reached_count++;
             break;
          }
       }
       if(tolerance_not_reached_count > 0)
       {
          break;
       }
    }

    // If the tolerance level has been reached, break out of the loop
    if(tolerance_not_reached_count == 0)
    {
       break;
    }
    // Update the temperature grid with the new values
    for(int i=0; i< NX; i++)
```

```
        {
            for(int j=0; j<NY; j++)
            {
                T[i][j] = T_new[i][j];
                //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
                //ofile  << T[i]<< std::endl;
                //ofile  << T[j]<< std::endl;
                //ofile  << T[i][j]<< std::endl;
                ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
                    //std::string combined = "T[" + std::to_string(i) + "][" + std::to_string(j) + "] =
" + std::to_string(T[i][j]) + "\n";
                //ofile << combined;
            }
        }
    }

    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds

    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;



    return 0;
}
```

COMPUTER CODE FOR POINT SUCCESSIVE OVER RELAXATION METHOD:

```
//PSOR
#include<iostream>
#include<math.h>
#include<fstream>
#include <ctime>
#define NX 21
#define NY 41

double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta;
double omega=1.5;
void Boundary_condition()
{
   for (int i=0; i<NX; i++)
   {
      T[i][0]= T_wall;
   }
   for (int i=0; i < NX; i++)
   {
      for (int j=0; j < (NY-1); j++)
      {
         T[i][j]=T_wall;
      }
```

```
    }
    for (int i=0; i < (NX-1); i++)
    {
        for (int j=0; j < NY; j++)
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < NX; i++)
    {
        T[i][NY-1]= T_top_wall;
    }
}

int main()
{
    clock_t start, end;
    double CPU_time_used;
    start = clock();
    delta_x = (LX/ NX);
    delta_y = (LY/ NY);
    beta = (delta_x / delta_y);

    std::cout << "Cell size in X axis: " << delta_x << std::endl;
    std::cout << "Cell size in Y axis: " << delta_y << std::endl;
    std::cout << "Beta: " << beta << std::endl;

    // Initialize the grid size
    for(int i=0; i< NX; i++)
```

```
    {
        for(int j=0; j<NY; j++)
        {
            T[i][j]=0;
        }
    }

    // first iteration begin
    for(int i=1; i< NX-1; i++)
    {
        for(int j=1; j<NY-1; j++)
        {
            T[i][j]=(      ((1-omega)*(T[i][j]))+      (omega/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
            //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
        }
    }

    Boundary_condition();

    //iteration start
    double tolerance = 0.01;
    double iteration = 0;
    double T_new[NX][NY];
    std::ofstream ofile("final_temp.txt");

    while(true)
    {
        iteration++;
```

```
    // Perform iterative calculations here
    for(int i=1; i< NX-1; i++)
    {
        for(int j=1; j<NY-1; j++)
        {
            T_new[i][j]=(                    ((1-omega)*(T[i][j]))                    +
(omega/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
        }
    }
    // Check if the desired tolerance level has been reached
    int tolerance_not_reached_count = 0;
    for(int i=1; i< NX-1; i++)
    {
        for(int j=1; j<NY-1; j++)
        {
            double delta_T = fabs(T_new[i][j] - T[i][j]);
            if(delta_T > tolerance)
            {
                tolerance_not_reached_count++;
                break;
            }
        }
        if(tolerance_not_reached_count > 0)
        {
            break;
        }
    }

    // If the tolerance level has been reached, break out of the loop
```

```
        if(tolerance_not_reached_count == 0)
        {
            break;
        }
        // Update the temperature grid with the new values
        for(int i=1; i< NX-1; i++)
        {
            for(int j=1; j<NY-1; j++)
            {
                T[i][j] = T_new[i][j];
                //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
                //ofile  << T[i]<< std::endl;
                //ofile  << T[j]<< std::endl;
                //ofile  << T[i][j]<< std::endl;
                ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
                        //std::string combined = "T[" + std::to_string(i) + "][" + std::to_string(j) + "] =
" + std::to_string(T[i][j]) + "\n";
            //ofile << combined;
            }
        }
    }
    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds
    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;
    return 0;  }
```

COMPUTER CODE FOR LINE SUCCESSIVE OVER RELAXATION METHOD:

```
//LSOR Gauss-sidel


#include<iostream>
#include<math.h>
#include<fstream>
#include <ctime>
#define NX 21
#define NY 41


double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta;
double omega = 1.5;
void Boundary_condition()
{
   for (int i=0; i<NX; i++)
   {
      T[i][0]= T_wall;
   }
   for (int i=0; i < NX; i++)
   {
      for (int j=0; j < (NY-1); j++)
      {
         T[i][j]=T_wall;
      }
   }
   for (int i=0; i < (NX-1); i++)
```

```
    {
        for (int j=0; j < NY; j++)
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < NX; i++)
    {
        T[i][NY-1]= T_top_wall;
    }
}

int main()
{
    clock_t start, end;
    double CPU_time_used;
    start = clock();
    delta_x = (LX/ NX);
    delta_y = (LY/ NY);
    beta = (delta_x / delta_y);

    std::cout << "Cell size in X axis: " << delta_x << std::endl;
    std::cout << "Cell size in Y axis: " << delta_y << std::endl;
    std::cout << "Beta: " << beta << std::endl;

    // Initialize the grid size
    for(int i=0; i< NX; i++)
    {
        for(int j=0; j<NY; j++)
        {
            T[i][j]=0;
        }
```

```
}

// first iteration begin
for(int j=1; j< NY-1; j++)
{
   T[0][j]=(                    ((1-omega)*(T[0][j]))                    +
(omega/(2*(1+(beta*beta))))*((T[1][j])+(beta*beta)*(T[0][j+1])+(beta*beta)*(T[0][j-1])) );
}

for(int i=1; i< NX-1; i++)
{
   T[i][1]=(  ((1-omega)*(T[i][1]))  +  (omega/(2*(1+(beta*beta))))*((T[i+1][1])+(T[i-
1][1])+(beta*beta)*(T[i][2])) );
    for(int j=2; j< NY-1; j++)
    {
      T[i][j]=(  ((1-omega)*(T[i][j]))  +  (omega/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
    }
   T[i][NY-2]=(  ((1-omega)*(T[i][NY-2]))  +  (omega/(2*(1+(beta*beta))))*((T[i+1][NY-
2])+(T[i-1][NY-2])+(beta*beta)*(T[i][NY-3])) );
}

for(int j=1; j< NY-1; j++)
{
   T[NX-1][j]=(  ((1-omega)*(T[NX-1][j]))  +  (omega/(2*(1+(beta*beta))))*((T[NX-
2][j])+(beta*beta)*(T[NX-1][j+1])+(beta*beta)*(T[NX-1][j-1])) );
}

Boundary_condition();

//iteration start
double tolerance = 0.01;
int iteration = 0;
double T_new[NX][NY];
```

```
std::ofstream ofile("final_temp.txt");


while(true)
{
   iteration++;


   // Perform iterative calculations here
   for(int j=1; j< NY-1; j++)
   {
      T_new[0][j]=(((1-omega)*(T[0][j]))
+(omega/(2*(1+(beta*beta))))*((T[1][j])+(beta*beta)*(T[0][j+1])+(beta*beta)*(T[0][j-1])) );
   }


   for(int i=1; i< NX-1; i++)
   {
      T_new[i][1]=(                    ((1-omega)*(T[i][1]))                    +
(omega/(2*(1+(beta*beta))))*((T[i+1][1])+(T[i-1][1])+(beta*beta)*(T[i][2])) );
      for(int j=2; j< NY-1; j++)
      {
         T_new[i][j]=(                    ((1-omega)*(T[i][j]))                    +
(omega/(2*(1+(beta*beta))))*((T[i+1][j])+(T[i-
1][j])+(beta*beta)*(T[i][j+1])+(beta*beta)*(T[i][j-1])) );
      }
      T_new[i][NY-2]=(                    ((1-omega)*(T[i][NY-2]))                    +
(omega/(2*(1+(beta*beta))))*((T[i+1][NY-2])+(T[i-1][NY-2])+(beta*beta)*(T[i][NY-3])) );
   }


   for(int j=1; j< NY-1; j++)
   {
      T_new[NX-1][j]=( ((1-omega)*(T[NX-1][j]))+ (omega/(2*(1+(beta*beta))))*((T[NX-
2][j])+(beta*beta)*(T[NX-1][j+1])+(beta*beta)*(T[NX-1][j-1])) );
   }
```

```
// Check if the desired tolerance level has been reached
int tolerance_not_reached_count = 0;
for(int i=0; i< NX; i++)
{
   for(int j=0; j<NY; j++)
   {
      double delta_T = fabs(T_new[i][j] - T[i][j]);
      if(delta_T > tolerance)
      {
         tolerance_not_reached_count++;
         break;
      }
   }
   if(tolerance_not_reached_count > 0)
   {
      break;
   }
}

// If the tolerance level has been reached, break out of the loop
if(tolerance_not_reached_count == 0)
{
   break;
}

// Update the temperature grid with the new values
for(int i=0; i< NX; i++)
{
   for(int j=0; j<NY; j++)
   {
      T[i][j] = T_new[i][j];
      //std::cout<< "T[" << i << "][" << j << "] = " << T[i][j] << std::endl;
```

```
            //ofile  << T[i]<< std::endl;
            //ofile  << T[j]<< std::endl;
            //ofile  << T[i][j]<< std::endl;
            ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
                  //std::string combined = "T[" + std::to_string(i) + "][" + std::to_string(j) + "] =
" + std::to_string(T[i][j]) + "\n";
            //ofile << combined;
          }
        }
    }

    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds

    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;

    return 0;
}
```

COMPUTER CODE FOR ADI METHOD WITHOUT RELAXATION FACTOR:

```
#include<iostream>
#include<math.h>
#include<fstream>
#define NX 21
#define NY 41

double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta;

void Boundary_condition()
{
   for (int i=0; i<NX; i++)
   {
      T[i][0]= T_wall;
   }
   for (int i=0; i < NX; i++)
   {
      for (int j=0; j < (NY-1); j++)
      {
         T[i][j]=T_wall;
      }
   }
   for (int i=0; i < (NX-1); i++)
```

```
    {
        for (int j=0; j < NY; j++)
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < NX; i++)
    {
        T[i][NY-1]= T_top_wall;
    }
}

int main()
{
     clock_t start, end;
    double CPU_time_used;
    start = clock();
    delta_x = (LX/ NX);
    delta_y = (LY/ NY);
    beta = (delta_x / delta_y);

    std::cout << "Cell size in X axis: " << delta_x << std::endl;
    std::cout << "Cell size in Y axis: " << delta_y << std::endl;
    std::cout << "Beta: " << beta << std::endl;

    // Initialize the grid size
    for(int i=0; i< NX; i++)
    {
        for(int j=0; j<NY; j++)
```

```
    {
        T[i][j]=0;
    }
}

Boundary_condition();

//iteration start
double tolerance = 0.01;
double iteration = 0.5; // Start with 0.5 for the first iteration
double T_new[NX][NY];

std::ofstream ofile("final_temp.txt");

while(true)
{
    // Perform ADI calculations for X direction
    for(int i=1; i< NX-1; i++)
    {
        for(int j=1; j< NY-1; j++)
        {
            T_new[i][j] = (1/(2*(1+beta*beta))) * ((T[i+1][j] + T[i-1][j] + beta*beta*(T[i][j+1]
+ T[i][j-1])));
        }
    }

    // Perform ADI calculations for Y direction
    for(int j=1; j< NY-1; j++)
    {
```

```
    for(int i=1; i< NX-1; i++)
    {
      T_new[i][j] = (1/(2*(1+beta*beta))) * ((T_new[i][j] + T_new[i][j-1] + beta*beta*(T_new[i+1][j] + T_new[i-1][j])));
    }
  }

  // Check if the desired tolerance level has been reached
  int tolerance_not_reached_count = 0;
  for(int i=0; i< NX; i++)
  {
    for(int j=0; j<NY; j++)
    {
      double delta_T = fabs(T_new[i][j] - T[i][j]);
      if(delta_T > tolerance)
      {
        tolerance_not_reached_count++;
        break;
      }
    }
    if(tolerance_not_reached_count > 0)
    {
      break;
    }
  }

  // If the tolerance level has been reached, break out of the loop
  if(tolerance_not_reached_count == 0)
  {
```

```
            break;
        }

        // Update the temperature grid with the new values
        for(int i=0; i< NX; i++)
        {
            for(int j=0; j<NY; j++)
            {
                T[i][j] = T_new[i][j];
                ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
            }
        }

        iteration += 0.5; // Increment the iteration by 0.5
    }

    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds

    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;

    return 0;
}
```

COMPUTER CODE FOR ADI METHOD WITH RELAXATION FACTOR:

```
//ADI WITH RELAXATION FACTOR


#include<iostream>
#include<math.h>
#include<fstream>
#define NX 21
#define NY 41


double LX = 1;
double LY = 2;
double T_top_wall = 100;
double T_wall = 0;
double T[NX][NY];
double delta_x, delta_y, beta, omega;


void Boundary_condition()
{
   for (int i=0; i<NX; i++)
   {
      T[i][0]= T_wall;
   }
   for (int i=0; i < NX; i++)
   {
      for (int j=0; j < (NY-1); j++)
      {
         T[i][j]=T_wall;
      }
```

```
    }
    for (int i=0; i < (NX-1); i++)
    {
        for (int j=0; j < NY; j++)
        {
            T[i][j]=T_wall;
        }
    }
    for (int i=0; i < NX; i++)
    {
        T[i][NY-1]= T_top_wall;
    }
}

int main()
{
    clock_t start, end;
    double CPU_time_used;
    start = clock();
    delta_x = (LX/ NX);
    delta_y = (LY/ NY);
    beta = (delta_x / delta_y);
    omega = 1.5; // Set the relaxation factor here

    std::cout << "Cell size in X axis: " << delta_x << std::endl;
    std::cout << "Cell size in Y axis: " << delta_y << std::endl;
    std::cout << "Beta: " << beta << std::endl;
    std::cout << "Omega: " << omega << std::endl;
```

```
// Initialize the grid size
for(int i=0; i< NX; i++)
{
   for(int j=0; j<NY; j++)
   {
      T[i][j]=0;
   }
}

// First iteration begin
for(int j=1; j< NY-1; j++)
{
   T[0][j]=( ((1-omega)*(T[0][j])) + (omega/(1+beta*beta))*(T[1][j]));
}

for(int i=1; i< NX-1; i++)
{
   T[i][1]=( ((1-omega)*(T[i][1])) + (omega/(1+beta*beta))*(T[i-1][1] + T[i+1][1]));
   for(int j=2; j< NY-1; j++)
   {
      T[i][j]=( ((1-omega)*(T[i][j])) + (omega/(1+beta*beta))*(T[i-1][j] + T[i+1][j] +
beta*beta*(T[i][j-1] + T[i][j+1])));
   }
   T[i][NY-2]=( ((1-omega)*(T[i][NY-2])) + (omega/(1+beta*beta))*(T[i-1][NY-2] +
T[i+1][NY-2] + beta*beta*(T[i][NY-3])));
}

for(int j=1; j< NY-1; j++)
{
   T[NX-1][j]=( ((1-omega)*(T[NX-1][j])) + (omega/(1+beta*beta))*(T[NX-2][j]));
```

```
}

Boundary_condition();

// Iteration start
double tolerance = 0.01;
double iteration = 0.5;
double T_new[NX][NY];

std::ofstream ofile("final_temp.txt");

while(true)
{
   iteration = iteration+0.5; // Update iteration level

   // Perform ADI calculations for X direction
   for(int i=1; i< NX-1; i++)
   {
      for(int j=1; j< NY-1; j++)
      {
         T_new[i][j]=(  ((1-omega)*(T[i][j]))  +  (omega/(1+beta*beta))*(T[i-1][j]  +
T[i+1][j]));
      }
   }

   // Perform ADI calculations for Y direction
   for(int j=1; j< NY-1; j++)
   {
      for(int i=1; i< NX-1; i++)
```

```
        {
            T[i][j] = T_new[i][j] + (omega/(1+beta*beta))*(T_new[i][j-1] + T_new[i][j+1] + beta*beta*(T_new[i-1][j] + T_new[i+1][j]));
        }
    }

    // Check if the desired tolerance level has been reached
    int tolerance_not_reached_count = 0;
    for(int i=0; i< NX; i++)
    {
        for(int j=0; j<NY; j++)
        {
            double delta_T = fabs(T_new[i][j] - T[i][j]);
            if(delta_T > tolerance)
            {
                tolerance_not_reached_count++;
                break;
            }
        }
        if(tolerance_not_reached_count > 0)
        {
            break;
        }
    }

    // If the tolerance level has been reached, break out of the loop
    if(tolerance_not_reached_count == 0)
    {
        break;
```

```
        }

        // Update the temperature grid with the new values
        for(int i=0; i< NX; i++)
        {
            for(int j=0; j<NY; j++)
            {
                T[i][j] = T_new[i][j];
                ofile << i << "\t" << j << "\t" << T[i][j] << std::endl;
            }
        }
    }

    std::cout << "Tolerance reached after " << iteration << " iterations." << std::endl;
    ofile.close();
    end = clock();
    CPU_time_used = static_cast<double>(end - start) / CLOCKS_PER_SEC;
    // Calculate the elapsed time in seconds

    std::cout << "CPU_time_used: " << CPU_time_used << " seconds" << std::endl;
    return 0;
}
```